POST-SILICON VALIDATION & DV PROJECT PORTFOLIO

Role: Lead Validation / Design Verification (DV) Engineer
Project: FPGA AXI-Lite Register Validation Engine & Serial Host Interface Framework
Target Platform: Xilinx Artix-7 FPGA (50 MHz System Clock, 115200 Baud UART)

---

1. Project Executive Summary

This project showcases a complete pre-silicon and post-silicon validation framework designed to verify an AXI-Lite Register Block on an Artix-7 FPGA. The system bridges external host-side serial UART communication with the FPGA's internal high-speed AXI-Lite bus, allowing automated software validation suites to run direct register read/write sweeps, toggle checks, and address decoding tests on physical silicon.

Key Technical Contributions

- **Pre-Silicon Verification**: Built SystemVerilog testbenches and AXI protocol checking monitors using Icarus Verilog (`iverilog` / `vvp`) to verify bus transactions, clock cycles (aligned to 50 MHz, 20 ns period), and asynchronous reset recovery.
- **Post-Silicon Automation**: Developed an automated Python regression runner (`run_regression.py`) with a byte-packet serial driver, communicating with the physical FPGA UART interface.
- **Coverage Instrumentation**: Engineered a coverage collection database tracking **Register Access Coverage**, **Access Policy Enforcement**, and **32-Bit Bit-Bash Toggle Coverage** (verifying every bit toggles both high '1' and low '0').
- **Reporting Infrastructure**: Authored HTML-based interactive logic-analyzer waveform simulators and live test metrics dashboards to present real-time verification status to team leads and stakeholders.

---

2. Subsystem Architecture Specifications

The register validation framework consists of 7 tightly integrated subsystems bridging physical serial communication with the internal AXI-Lite bus lines:

Subsystems Breakdown:

1. **UART Transceivers (`uart_rx` & `uart_tx`)**: Converts raw serial lines to 8-bit parallel byte frames. Includes a 16x oversampling clock generator on the RX line to filter out line jitter and ensure robust clock synchronization.
2. **UART Register Interface (`uart_register_interface`)**: Deserializes opcodes, addresses, and data from multi-byte serial packets. Generates response packets (status bytes + read data) back to the serial transmitter.
3. **AXI-Lite Master Cmd IF (`axi_lite_master_cmd_if`)**: Serves as the AXI master node. Automatically executes read/write handshake cycles (AW, W, B, AR, R channels) on the bus. Implements a 1024-cycle watchdog timer to abort stalled transactions and protect the system.
4. **AXI-Lite Register Block (`axi_lite_register_block`)**: Implements 12 addressable hardware registers across three logical blocks:
5. **GPIO Block (Base 0x00)**: `GPIO_OUT` (RW, drives board LEDs), `GPIO_IN` (RO, slide switches), `GPIO_DIR` (RW, direction mask), `SCRATCH` (RW, general-purpose scratchpad).
6. **Status Block (Base 0x100)**: `DEVICE_ID` (RO, static identifier `0xA5A50009`), `STATUS_FLAGS` (RO, latches bus errors), `ERROR_CLEAR` (WO, error latch clearing), `VERSION` (RO, firmware version).
7. **Timer Block (Base 0x200)**: `TIMER_CTRL` (RW, counter enable/reload), `TIMER_COUNT` (RO, live counter), `TIMER_LIMIT` (RW, threshold), `TIMER_STATUS` (RO, expiration interrupts).
8. **AXI-Lite Protocol Checker (`axi_lite_protocol_checker`)**: Monitors bus signals passively to flag violations of the AXI specification (e.g. handshake signals dropping before validation, invalid response values).
9. **Error Injector (`axi_lite_error_injector`)**: Forces diagnostic bus faults (unmapped addresses, unaligned register access, write-to-read-only blocks) to test the robustness of the slave response decoder.
10. **Register Access Monitor (`register_access_monitor`)**: Captures real-time AXI transactions, compiling statistical records of block accesses.

---

3. Validation Strategy & Test Matrix

A structured 7-part regression test suite was developed to validate design specifications:

Test Case ID	Test Name	Target Specifications	Pass Criteria

TC-01	Reset Values Verification	Checks all 12 registers against spec default values immediately after reset.	All registers match target reset values (e.g. `DEVICE_ID = 0xA5A50009`, control registers = `0x00`).
TC-02	Write/Read Back RW Registers	Exercises read/write paths for all Read/Write (RW) registers.	Written data is stored and read back identically with no corruption.
TC-03	Read-Only Write Protection	Checks write dropping on Read-Only (RO) registers.	Writes to RO registers (e.g. `VERSION`, `GPIO_IN`) are dropped; register values remain unchanged.
TC-04	Write-Only Policy Check	Validates Write-Only (WO) register configurations.	Writes to `ERROR_CLEAR` execute successfully; reads return zero.
TC-05	32-Bit Toggle Bit-Bash Sweep	Performs walking 1s and walking 0s sweep across all RW bits.	Every individual bit in RW registers successfully toggles between 0 and 1.
TC-06	Out-of-Bounds Address Decoding	Checks bus behavior for illegal address transactions.	Requests to unmapped addresses receive `DECERR` responses without locking up the bus.
TC-07	Reset Recovery Check	Triggers soft reset commands and checks configuration recovery.	System returns to initial defaults; error flag registers are cleared.

---

4. Verification Methodologies

A. Pre-Silicon Verification (SystemVerilog / Verilog RTL)

- **Simulation Engine**: Utilized Icarus Verilog (`iverilog` / `vvp`) to compile and run simulation captures.
- **Clock Domain Constraints**: Constrained the main system clock to **50 MHz (20.00 ns period)**, matching the target FPGA's hardware specifications.
- **Assertive Verification**: Embedded behavioral assertion checks inside testbenches to verify AXI handshakes: $$\text{AWVALID} \land \text{AWREADY} \implies \text{Address Latch}$$ $$\text{WVALID} \land \text{WREADY} \implies \text{Data Write}$$ $$\text{ARVALID} \land \text{ARREADY} \implies \text{Address Read}$$

B. Post-Silicon Validation (Artix-7 FPGA Silicon)

- **Serial Packet Framing**: Serial packets are encapsulated in multi-byte framing structures to avoid data framing slips:
- **Write Command Frame (9 Bytes)**: `[0x55 (Sync)] [0x01 (Write Op)] [Address (4 Bytes)] [Data (3 Bytes)]`
- **Write Response Frame (6 Bytes)**: `[0xAA (Sync)] [0x01 (Op)] [Status (1 Byte)] [Padding (3 Bytes)]`
- **Read Command Frame (5 Bytes)**: `[0x55 (Sync)] [0x02 (Read Op)] [Address (3 Bytes)]`
- **Read Response Frame (6 Bytes)**: `[0xAA (Sync)] [0x02 (Op)] [Status (1 Byte)] [Data (3 Bytes)]`
- **Python Regression Environment**: Runs automated validation test cases sequentially over the UART USB interface, capturing raw response packets and evaluating pass/fail status.

---

5. Coverage Instrumentation & Metrics

To ensure verification completeness, the regression engine aggregates three metrics:

1. **Register Access Coverage**: Tracks whether every register has been successfully read and written at least once during regression: $$\text{Access Coverage} = \frac{\text{Accessed Registers}}{\text{Total Spec Registers}} \times 100\%$$
2. **Access Policy Coverage**: Verifies that register permissions are enforced (e.g. confirming that attempts to write to RO registers fail).
3. **Bit Toggle Coverage**: Monitors individual register bits. A bit is toggled if it has been written to `1` and written to `0` during verification: $$\text{Bit Coverage} = \frac{\text{Bits Toggled High} +

\text{Bits Toggled Low}}{2 \times \text{Total Register Bits}} \times 100\%$$

Current Regression Status:

- **Total Tests Run**: 7 / 7 Passed
- **Register Access Coverage**: 100.0%
- **Bit Toggle Coverage**: 100.0% (Verified 160/160 active RW bits toggled high and low)

---

6. Premium Interactive Deliverables

A core feature of this framework is the visualization of validation metrics for engineering reviews:

1. **Interactive Regression Report (`report.html`)**: Summarizes test suite outcomes with accordion drawers showing verification console traces and dynamic SVG progress circles.
2. **Post-Silicon Validation Dashboard (`dashboard.html`)**: Displays a cell grid representing the 32 bits of each RW register. Cells are color-coded (green = fully toggled, amber = partially toggled, red = untoggled) and provide custom tooltips on hover detailing bitfield descriptions. Contains a "Run Validation Sweep" button that runs a live simulation of a hardware verification sweep.
3. **Logic Analyzer Waveform Viewer**: An SVG logic analyzer panel embedded in the specification document. Simulates write/read clock cycles and displays causality arrows (Bezier dependency lines) between valid/ready signals, illustrating AXI bus dependencies in real-time.

---

7. Mock Interview Questions & Validation Scenarios

(Use these to prepare for technical validation interviews)

Q1: How did you handle clock domain differences between the host computer and the FPGA?

> **Answer**: The host runs on an asynchronous clock relative to the FPGA. We bridged this by implementing a 16x oversampling UART receiver on the FPGA. It samples the serial line in the middle of each bit time-slot, filters out high-frequency noise using digital voting logic, and asserts a single-cycle valid pulse once a frame is aligned. This decouples the transmission jitter from the internal 50 MHz AXI-Lite bus.

Q2: What happens if a slave register fails to assert its READY signal? How does your system recover?

> **Answer**: If a slave lockup occurs, the AXI-Lite Master Cmd IF's watchdog timer expires after 1024 clock cycles. The master automatically aborts the transaction, returns a `SLVERR` response status to the host, and latches a timeout flag in the `STATUS_FLAGS` register. This prevents the entire FPGA interconnect from hanging.

Q3: How do you verify that the write protection on your Read-Only (RO) registers is actually working?

> **Answer**: In TC-03, we execute write commands containing walking pattern payloads (e.g. `0x55555555`, `0xAAAAAAAA`) to RO registers like `VERSION` or `DEVICE_ID`. We then read them back. If the read-back value matches the hardcoded default rather than the payload, the write drop is verified. Additionally, the AXI Protocol Checker ensures that the bus transaction responds with `OKAY` (rather than locking up) while dropping the write data.